



Lecture Twelve

Run-Time Type Identification (RTTI) and Casting Operator

Ref: Herbert Schildt, Teach Yourself C++, Third Edⁿ (Chapter 12)



Run-Time Type Identification (RTTI)

- In C, there is **no need** for **run-time type identification** because the **type** of each object is known at **compile time**.
- In **polymorphic** language C++, there can be **situations** in which the **type of an object** is **unknown** at **compile time**.
- To **obtain an object's type**, need to include **<typeinfo>** and use **typeid** as follows: **typeid(object)**
- The class **type_info** defines the following **public members**:
 - Bool operator == (const type_info &ob);
 - Bool operator != (const type_info &ob);
 - Bool before (const type_info &ob);
 - const char *name();



Run-Time Type Identification (RTTI)

```
#include <iostream>
#include <typeinfo>
using namespace std;
```

```
class base {
    virtual void f();
}
```

```
Class derived1::public base {
    //.....
}
```

```
Class derived2::public base {
    //.....
}
```

```
int main(){
    int i;
    base *p, b_ob;
    derived1 ob1;
    derived2 ob2;

    cout << typeid(i).name() << endl;

    p = &b_ob;
    cout << typeid(*p).name() << endl;

    p = &ob1;
    cout << typeid(*p).name() << endl;

    p = &ob2;
    cout << typeid(*p).name() << endl;

    return 0;
}
```



Type Casting Operator

➤ There are **four types** of casting: **dynamic_cast**, **const_cast**, **reinterpret_cast** and **static_cast**.

➤ The **general form** of **dynamic_cast** is as follows:

dynamic_cast<target-type>(exp);

Example:

```
base *bp, b_ob;  
derived *dp, d_ob;
```

```
bp = &d_ob;  
dp = dynamic_cast<derived*>(bp); //Ok
```

```
bp = &b_ob;  
dp = dynamic_cast<derived*>(bp); //Error
```

➤ The **dynamic cast** can be used to **cast one type of pointer into another** or **one type of reference into another**.

➤ If the **dynamic_cast** involves **pointer**, it returns **NULL** if the **casting fails**.

➤ If the **dynamic_cast** involves **reference type fail**, it throws **bad_cast** exception

➤ The **dynamic_cast** for template or general class is as follows:

```
p2 = dynamic_cast<square<double*>>(p1);
```



Type Casting Operator

- The **general forms** of **different casting** are as follows:

```
const_cast<target-type>(exp);  
reinterpret_cast<target-type>(exp);  
static_cast<target-type>(exp);
```

- The **reinterpret_cast** operator changes **one pointer type into another pointer type**. It can change **a pointer into an integer** and **an integer into a pointer**.

```
int i;  
char *p = "This is a string.";  
i = reinterpret_cast<int>(p); //cast pointer to integer
```

- The **const_cast** operator is used to **explicitly override const and/or volatile** in a cast.

```
void f(const int *p){  
    int *v;  
    v = const_cast<int *>(p); // cast away const-ness  
    *v = 100;                // modify the object through v  
}
```

- The **static_cast** operator performs a **non-polymorphic cast**. For example, it can be used to cast **a base class pointer to a derived class pointer**.

```
float f = 199.22;  
int i = static_cast<int>(f);
```